

Informe tecnico: Real-Time Finger Detection

Proyecto Python con OpenCV y MediaPipe para contar dedos levantados en tiempo real

Fecha: 20/05/2026 | Entorno: Windows | Proyecto: Real-Time Finger Detection

Resumen: La aplicacion captura video de la camara, detecta una o dos manos con MediaPipe, calcula los dedos extendidos a partir de los 21 landmarks y muestra el resultado sobre el video en vivo.

1. Objetivo del proyecto

El objetivo fue desarrollar una aplicacion de vision por computadora capaz de contar, en tiempo real, cuantos dedos estan levantados en una mano usando la camara del dispositivo. Durante la implementacion se amplio el soporte para detectar hasta dos manos.

- Captura de video en vivo mediante OpenCV.
- Deteccion de manos con MediaPipe Hands.
- Conteo de dedos extendidos de 0 a 5 por mano.
- Overlay con landmarks, FPS y numero de dedos detectados.
- Suavizado temporal para reducir parpadeos en el conteo.

2. Tecnologias utilizadas

Tecnologia	Uso en el proyecto
Python	Lenguaje principal de la aplicacion y scripts de soporte.
OpenCV	Captura de webcam, transformacion de frames y visualizacion en ventana.
MediaPipe	Deteccion de mano y extraccion de 21 landmarks.
NumPy	Dependencia numerica usada por OpenCV y MediaPipe.
python-docx	Generacion de este informe tecnico en formato Word.

3. Estructura del proyecto

Archivo	Responsabilidad
main.py	Punto de entrada. Gestiona argumentos, camara, ventana, overlay y controles.
hand_detector.py	Encapsula MediaPipe, logica de dedos, dibujo de landmarks y soporte para recursos.
utils.py	Incluye FPSCounter y StableCounter para metricas y estabilizacion.
requirements.txt	Fija versiones compatibles de OpenCV, MediaPipe y NumPy.

README.md	Instrucciones de instalacion, ejecucion y controles.
-----------	--

4. Funcionamiento general

1. Se abre la camara seleccionada con OpenCV y se configura resolucion y FPS objetivo.
2. Cada frame se invierte horizontalmente por defecto para una experiencia tipo espejo.
3. El frame BGR se convierte a RGB y se procesa con MediaPipe Hands.
4. Si hay manos detectadas, se extraen landmarks y handedness.
5. Para indice, medio, anular y menique se compara la coordenada Y de punta, PIP y MCP.
6. Para el pulgar se compara la apertura lateral teniendo en cuenta si la mano es Left o Right.
7. El conteo bruto se suaviza con una ventana temporal corta y se muestra en pantalla.

5. Logica de conteo de dedos

MediaPipe devuelve 21 puntos normalizados de la mano. La regla principal interpreta un dedo como levantado cuando la punta esta por encima de sus articulaciones intermedias. Esta aproximacion funciona mejor con la palma visible y los dedos orientados hacia arriba.

Dedo	Landmark punta	Comparacion principal
Pulgar	4	Apertura lateral frente a landmarks 3 y 2, ajustada por mano izquierda/derecha.
Indice	8	$tip.y < pip.y < mcp.y$
Medio	12	$tip.y < pip.y < mcp.y$
Anular	16	$tip.y < pip.y < mcp.y$
Menique	20	$tip.y < pip.y < mcp.y$

Estabilidad: El conteo final mostrado usa la moda de los ultimos frames. Esto evita saltos rapidos cuando un dedo queda cerca del limite entre levantado y doblado.

6. Instalacion y ejecucion

El proyecto quedo preparado con un entorno virtual local en la carpeta .venv.

```
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python main.py
```

Ejecucion directa sin activar el entorno:

```
.\.venv\Scripts\python.exe main.py
```

Opciones utiles:

```
python main.py --hands 1
```

```
python main.py --hands 2
python main.py --dominant-hand Right
python main.py --thumb-mode flip
```

7. Controles de usuario

Tecla	Accion
Q / ESC	Cerrar la aplicacion.
R	Reiniciar la ventana de estabilidad del conteo.
H	Invertir la regla del pulgar si la camara lo cuenta al revés.

8. Incidencias encontradas y resolucion

Incendencia	Causa	Solucion aplicada
mediapipe no exponia mp.solutions	La version mas reciente instalada no mantenía la API clásica esperada.	Se fijo mediapipe==0.10.21 en requirements.txt.
Conflicto NumPy/OpenCV	opencv-python moderno solicitaba NumPy 2 mientras MediaPipe clásico requiere NumPy 1.x.	Se uso opencv-contrib-python==4.11.0.86 y numpy>=1.26,<2.
Fallo de recursos por ruta con acento	MediaPipe en Windows fallo cargando grafos desde la antigua ruta DeteccionDedos con caracter acentuado.	Se redirigieron los recursos de MediaPipe a una carpeta temporal ASCII antes de iniciar Hands.

9. Pruebas realizadas

Prueba	Resultado
Compilacion de modulos Python	Correcta: main.py, hand_detector.py y utils.py compilan sin errores.
Import de dependencias	Correcto: OpenCV 4.11.0, MediaPipe 0.10.21 y NumPy 1.26.4.
Inicializacion del detector	Correcta: HandDetector se crea y se cierra sin excepciones.
Captura de camara	Correcta: la webcam se abre y entrega frames de 480x640.
Prueba funcional del usuario	Confirmada: la aplicacion funciona correctamente y detecta manos en vivo.

10. Limitaciones y mejoras futuras

- La logica funciona mejor con palma visible, buena iluminacion y dedos orientados hacia arriba.

- El pulgar depende de la clasificacion Left/Right y puede requerir la tecla H segun la camara.
- Se podria anadir calibracion guiada para adaptar la postura inicial de cada usuario.
- Se podria guardar historico de FPS y precision estimada para sesiones de prueba.
- Se podria empaquetar como ejecutable para equipos sin entorno Python preparado.

11. Conclusiones

El proyecto cumple los requisitos planteados: usa camara en tiempo real, detecta manos, identifica dedos levantados, muestra el conteo sobre el video y mantiene una estructura clara por modulos. Tras los ajustes de dependencias y compatibilidad de rutas en Windows, la aplicacion quedo funcional y preparada para ampliaciones.

Anexo A. Dependencias fijadas

opencv-contrib-python==4.11.0.86

mediapipe==0.10.21

numpy>=1.26.0,<2